

Assignment 3

Deployment and Integration of AI Agents

Student Name: RenXuan

Student ID: ZY2557207

Website Integration: RenXuan's blog / Assignment 3 / DeepSeek AI Q&A platform

Objective

This assignment practices deploying and using AI agents in a development workflow. The submitted work includes an online DeepSeek Q&A platform, a local-model deployment workflow with Ollama, IDE-assisted code refactoring, and reflective documentation comparing online and local model usage.

Rubric Checklist

Requirement	Implementation	Status
Online Agent	A static web Q&A platform calls the DeepSeek OpenAI-compatible chat-completions API. It also supports pasted or uploaded text context for file analysis.	Completed
Local Model Deployment	Ollama setup commands and a local interaction script are documented for models such as <code>qwen2.5:7b</code> .	Documented workflow
IDE Integration	An AI coding assistant was used to refactor a Python delay-analysis function and verify that the output stayed the same.	Completed
Documentation and Reflection	The report records setup steps, challenges, model comparison, and workflow reflection.	Completed

Online Agent: DeepSeek Q&A Platform

The online agent is implemented as `site/deepseek_qa.html` with JavaScript in `site/assets/deepseek_qa.js`. The page asks the user to enter a DeepSeek API Key in the

browser. The key is not hard-coded in the source files, which avoids exposing secrets when uploading the project to GitHub.

The JavaScript request uses the DeepSeek chat-completions endpoint:

```
POST https://api.deepseek.com/chat/completions
Authorization: Bearer <DeepSeek API Key>
Content-Type: application/json
```

The page supports model selection for `deepseek-v4-flash`, `deepseek-v4-pro`, and legacy compatibility options such as `deepseek-chat` and `deepseek-reasoner`. It also includes a context text area and a text-file upload input, so the agent can answer questions based on pasted course notes, code, Markdown, or other text files.

Local Model Deployment with Ollama

The local deployment workflow uses Ollama. The intended commands are:

```
ollama pull qwen2.5:7b
ollama serve
python scripts\ollama_local_agent.py --model qwen2.5:7b --prompt "Explain
what an AI coding agent does in three bullet points."
```

In the checked workspace, Ollama was not running, so the local script prints setup guidance instead of failing silently. This makes the deployment process reproducible once Ollama is installed.

IDE Integration

The AI assistant was used to refactor a Python function from `avg_delay` to `average_delay`. The refactor improved naming, type hints, missing-value handling, and direct script execution while preserving the same result.

```
python scripts\refactor_before.py
python scripts\refactor_after.py
python -c "import ast, pathlib; [ast.parse(path.read_text(encoding='utf-
```

```
8')) for path in pathlib.Path('scripts').glob('*.py')]; print('syntax ok')"
```

Online vs. Local Model Comparison

Dimension	Online Model	Local Model
Setup	Requires API key and endpoint configuration.	Requires Ollama installation and model download.
Performance	Usually stronger for complex reasoning and writing.	Depends on local hardware and model size.
Privacy	Input is sent to a remote API provider.	Data can remain on the local computer.
Cost	May consume API quota or paid tokens.	No per-call API billing after download.
Best Use	Complex coding help, summarization, polished answers.	Private notes, offline experiments, quick drafts.

Challenges and Solutions

- **API key security:** the key is entered in the browser and not committed to GitHub.
- **Static GitHub Pages limitation:** the page is front-end only; if a browser blocks API requests, a local proxy or Python script can be used for demonstration.
- **Ollama availability:** the local script reports clear installation and model-pull steps when the service is unavailable.
- **Workflow verification:** Python scripts were syntax-checked and refactor examples were run before submission.

Reflection

Online models are the fastest way to build a useful agent because the model capability is provided by the API. They are suitable for complex reasoning, file analysis, and code explanation. The main concern is privacy and API-key management.

Local models require more setup, but they give better control over privacy and offline use. A practical workflow is to use online models for difficult reasoning and local models for private

drafts or experiments. IDE integration is most useful when the prompt is specific and the result is verified with tests or command-line checks.

References

1. DeepSeek API Documentation: <https://api-docs.deepseek.com/>
2. Ollama Windows Documentation: <https://docs.ollama.com/windows>
3. Ollama Windows Download: <https://ollama.com/download/windows>